Reviewer Pack — Minimal Rank of Quadratic Forms and Resolution Proof Length

Entry 1c61ebbfe36c · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-21 23:12:18 UTC

Minimal Rank of Quadratic Forms and Resolution Proof Length

Entry ID: `1c61ebbfe36c` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-05-21 23:12:18 UTC

1. Conjecture

Field A (mathematical branch): Algebraic Geometry: Quadratic Forms **Field B** (complexity object): Complexity Theory: Resolution Proof Complexity

Statement:

> ['For every Boolean formula in conjunctive normal form (CNF) with n variables, the minimal rank of its associated quadratic form is linearly related to the length of a shortest resolution proof for that CNF.', 'More precisely, there exists a constant c such that for all CNFs, the minimal rank of the quadratic form is at most c times the length of any resolution proof for the CNF.', 'This relationship holds for all instances with n ≤ 40.']

Rationale (proposer's reasoning):

> ['Quadratic forms provide a way to encode Boolean functions in an algebraic setting. The minimal rank of a quadratic form could capture essential properties of the function that are relevant to its complexity, such as the length of a resolution proof.', 'Since resolution is a key algorithm for proving satisfiability, any relationship between the rank of quadratic forms and the proof length could offer insights into the complexity of SAT.']

Taxonomy category: `AlgebraicGeometry_ResolutionProofComplexity` (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `fb1b5569877f9177`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> A conjecture is supported if, across all CNFs with n ≤ 40 variables, the ratio of the average minimal rank to the average resolution proof length is less than or equal to a constant c.

3. Barrier filter (F1)

Final: `PASS`

Barrier	Verdict	Confidence	LLM₁	LLM₂
RELATIVIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	UNCERTAIN	1.00	UNCERTAIN	HITS
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - Minimal Rank Quadratic Forms AND Resolution Proof Length - Quadratic Forms in Algebraic Geometry AND Resolution Complexity Theory - Conjunctive Normal Form AND Quadratic Form Rank Resolution Proof

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=1, elapsed=0.1s

5.1 Generated Python source

```python
import random
import math
from fractions import Fraction

def gaussian_elimination(matrix):
    n = len(matrix)
    for i in range(n):
        max_row = i
        for j in range(i+1, n):
            if abs(matrix[j][i]) > abs(matrix[max_row][i]):
                max_row = j
        matrix[i], matrix[max_row] = matrix[max_row], matrix[i]
        if matrix[i][i] == 0:
            continue
        denom = Fraction(1, matrix[i][i])
        for j in range(i, n):
            matrix[i][j] *= denom
        for k in range(n):
            if k != i and matrix[k][i] != 0:
                factor = -matrix[k][i]
                for j in range(i, n):
                    matrix[k][j] += factor * matrix[i][j]

def quadratic_form_rank(CNF):
    n = len(CNF)
    Q = [[0] * n for _ in range(n)]
    for clause in CNF:
        for lit1 in clause:
            i = abs(lit1) - 1
            for lit2 in clause:
                j = abs(lit2) - 1
                if lit1 > 0 and lit2 > 0:
                    Q[i][j] += 1
```

```
    gaussian_elimination(Q)
    rank = sum(1 for row in Q if any(row))
    return rank

def resolution_proofs(CNF):
    clauses = set(tuple(sorted(clause)) for clause in CNF)
    proof = []
    while True:
        new_clause = None
        for clause1 in clauses:
            for clause2 in clauses:
                if len(set(clause1) & set(clause2)) == 1:
                    lit1, lit2 = next(iter(set(clause1) ^ set(clause2)))
                    if lit1 > 0 and -lit2 in clause1:
                        new_clause = tuple(sorted([x for x in clause1 if x != -lit2] + [x for x in
                        break
            if new_clause:
                break
        if not new_clause:
            return proof
        clauses.add(new_clause)
        proof.append(new_clause)

def run_trial(seed: int) -> dict:
    random.seed(seed)
    n = 40
    instances_tested = 0
    total_rank = 0
    total_length = 0

    for _ in range(30):
        CNF = [[random.randint(-n, -1), random.randint(1, n)] for _ in range(random.randint(5, 20)
        rank = quadratic_form_rank(CNF)
        proof_length = len(resolution_proofs(CNF))
        total_rank += rank
        total_length += proof_length
        instances_tested += 1

    mean_rank = Fraction(total_rank, instances_tested)
    mean_length = Fraction(total_length, instances_tested)
    conjecture_holds = mean_rank <= 2 * mean_length
    counterexample = "" if conjecture_holds else "mapping_undefined"

    return {
        "metric_name": "Minimal Rank / Resolution Proof Length",
        "metric_value": float(mean_rank),
        "instances_tested": instances_tested,
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

if __name__ == "__main__":
    import sys
    seeds = [int(x) for x in sys.argv[1:]] or [2**i + 1 for i in range(5, 8)]
    results = []
    for seed in seeds:
```

```
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_rank = sum(r["metric_value"] for r in results) / len(results)
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_rank} std=0.0 support_fraction=1.0")
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_rank} std=0.0 support_fraction={support_fraction}")
    else:
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conj
        print(f"RESULT: FALSIFIED counterexample=\"mapping_undefined\" first_failing_seed={first_f
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_fb19d56b.py", line 103, in <module>
    result = run_trial(seed)
             ^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_fb19d56b.py", line 79, in run_trial
    rank = quadratic_form_rank(CNF)
           ^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_fb19d56b.py", line 46, in quadratic_for
    Q[i][j] += 1
    ~^^^
IndexError: list index out of range
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

> The test code crashed before producing data, which prevents us from verifying the conjecture's support condition. | next: Re-run the test with proper error handling to collect data for verification.

11. Audit log (LLM calls)

Total LLM calls: 11

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	15966
2	propose	ollama_remote	glm4:latest	0	0	18305
3	propose	ollama_remote	glm4:latest	0	0	17129
4	preregistration	ollama_remote	glm4:latest	0	0	9503
5	novelty	ollama_remote	glm4:latest	0	0	8264
6	novelty	ollama_remote	glm4:latest	0	0	9097
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	17055
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	26219
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	15952
10	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13224
11	judge	ollama_remote	glm4:latest	0	0	55492

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 206206 ms total latency. Provider mix: {'ollama_remote': 11}

(full prompt+response transcripts available in `research/audit/1c61ebbfe36c.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/1c61ebbfe36c.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/1c61ebbfe36c.tar.gz` (if generated)